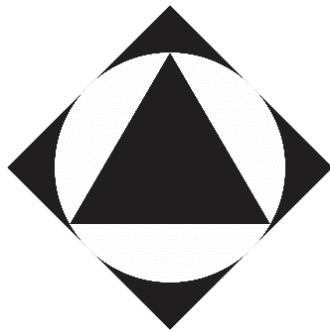


Platform Lapor online berbasis WEBGIS

KOMPUTASI AWAN

ETS 2



Disusun oleh:

AGIEL FERNANDA PUTRA

15-2022-032

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INDUSTRI
INSTITUT TEKNOLOGI NASIONAL
BANDUNG 2026**

BAB I

PENDAHULUAN

1.1. Latar Belakang

Adanya permasalahan pada infrastruktur bisa membuat suatu mobilitas masyarakat terganggu seperti, kerusakan jalan, kemacetan tidak terduga, hingga kecelakaan lalu lintas sering kali terjadi dan minim pengawasan secara langsung (real-time).

Untuk mengatasi hal tersebut, diperlukan sebuah sistem berbasis Sistem Informasi Geografis (GIS), di mana masyarakat dapat berpartisipasi aktif dalam melaporkan kondisi riil di lapangan. Sistem ini harus mampu memetakan lokasi insiden secara presisi, menyimpan bukti visual, serta dapat dikelola oleh pihak berwenang secara efisien.

1.2. Deskripsi aplikasi

Aplikasi web interaktif berbasis Sistem Informasi Geografis (GIS) yang memfasilitasi pelaporan insiden infrastruktur dan lalu lintas (seperti kecelakaan, kemacetan, dan jalan rusak). Aplikasi ini mengusung konsep pemetaan partisipatif, di mana pengguna dapat menandai titik koordinat presisi dari lokasi kejadian langsung di atas peta interaktif. Setiap laporan yang dikirim wajib menyertakan deskripsi dan unggahan foto bukti kejadian. Seluruh titik pelaporan kemudian akan divisualisasikan dalam bentuk pemetaan kluster, sehingga memudahkan proses validasi dan pemantauan distribusi insiden secara menyeluruh.

1.3. Target Pengguna

Sistem GeoLapor dirancang untuk melayani dua jenis target pengguna utama (keduanya), yaitu:

1. Masyarakat (Publik): Berperan sebagai kontributor data aktif. Masyarakat dapat melakukan registrasi, mengamati sebaran insiden di sekitarnya melalui peta interaktif, dan berpartisipasi langsung dalam melaporkan insiden dengan menentukan titik koordinat serta mengunggah foto bukti kejadian.
2. Admin (Pihak Berwenang / Pengelola): Berperan sebagai verifikator dan pengawas sistem. Admin memiliki hak akses (*privilege*) penuh terhadap *dashboard* pengelola

untuk memantau, memvalidasi (menerima atau menolak laporan), serta mengelola seluruh basis data insiden dan pengguna.

BAB II

LANDASAN TEORI

2.1 Cloud Computing (Komputasi Awan)

Cloud computing atau komputasi awan adalah model penyampaian layanan komputasi termasuk peladen (*server*), ruang penyimpanan (*storage*), basis data, jaringan, dan perangkat lunak melalui internet ("awan"). Model ini memungkinkan penggunaanya untuk menyewa sumber daya teknologi informasi secara dinamis tanpa perlu membangun infrastruktur fisik sendiri. Konsep utama dari komputasi awan adalah skalabilitas, fleksibilitas, dan efisiensi biaya yang berbasis pada penggunaan.

2.2 Google Cloud Platform (GCP)

Google Cloud Platform (GCP) merupakan platform komputasi awan komprehensif yang disediakan oleh Google, yang menawarkan berbagai layanan infrastruktur TI secara global. Dalam pengembangan sistem pelaporan spasial ini, arsitektur yang disyaratkan diimplementasikan menggunakan layanan GCP sebagai padanan dari ekosistem AWS.

2.2.1 Google Virtual Private Cloud (VPC) dan Subnet

Google VPC memungkinkan pembuatan jaringan virtual yang terisolasi secara logis di dalam infrastruktur Google. Jaringan ini dikonfigurasi ke dalam sistem *subnetting* untuk memisahkan lalu lintas publik dan privat. *Public Subnet* digunakan untuk menangani akses internet yang masuk, sedangkan *Private Subnet* digunakan untuk mengamankan komponen sensitif seperti basis data (*database*) dan komputasi aplikasi agar tidak terpapar langsung oleh jaringan luar.

2.2.2 Docker, Google Artifact Registry (GAR), dan Google Cloud Run

Docker adalah platform untuk membungkus kode aplikasi beserta dependensinya ke dalam unit mandiri bernama *container*. Dalam ekosistem GCP, *image* dari *Docker* tersebut disimpan pada layanan Google Artifact Registry (sebagai ekuivalen dari Amazon ECR). Selanjutnya, *container* tersebut dijalankan dan dikelola secara *serverless* menggunakan Google Cloud Run (sebagai ekuivalen dari Amazon ECS), yang secara otomatis melakukan penskalaan berdasarkan jumlah permintaan pelaporan (*traffic*) yang masuk.

2.2.3 Google Cloud SQL

Google Cloud SQL adalah layanan basis data relasional yang terkelola penuh (ekuivalen dari Amazon RDS). Pada sistem ini, *Cloud SQL* ditempatkan di dalam *Private Subnet* VPC untuk mengamankan data pengguna dan data pelaporan. Mesin basis data yang digunakan pada *Cloud SQL* ini adalah PostgreSQL, yang diintegrasikan dengan ekstensi PostGIS untuk memproses dan menyimpan koordinat data spasial secara efisien.

2.2.4 Google Cloud Storage (GCS) dan Cloud CDN

Google Cloud Storage (GCS) adalah layanan penyimpanan objek yang skalabel dan aman. GCS digunakan sebagai media penyimpanan (*storage*) utama untuk menampung *file*

gambar atau foto bukti laporan yang diunggah oleh masyarakat, berfungsi sebagai padanan dari Amazon S3. Untuk mendistribusikan *file* foto tersebut ke pengguna dengan latensi yang sangat rendah, sistem menggunakan Google Cloud CDN (ekuivalen dari Amazon CloudFront). Cloud CDN akan melakukan proses *caching* terhadap konten dari GCS dan mendistribusikannya melalui *edge network* global milik Google, sehingga menghindari akses langsung ke *bucket* penyimpanan yang dilarang dalam ketentuan sistem.

2.3 Continuous Integration dan Continuous Deployment (CI/CD)

Penerapan *CI/CD pipeline* pada proyek ini memanfaatkan GitHub Actions untuk mengotomatisasi proses integrasi dan *deployment* kode sumber (*source code*) yang disimpan di GitHub. Setiap kali pengembang melakukan komit pembaruan, *GitHub Actions* akan mengeksekusi proses *build container image* baru, mendorongnya (*push*) ke *Google Artifact Registry*, dan secara otomatis melakukan *deployment* pembaruan tersebut ke *Google Cloud Run*.

2.4 Sistem Informasi Geografis (GIS) dan Data Spasial

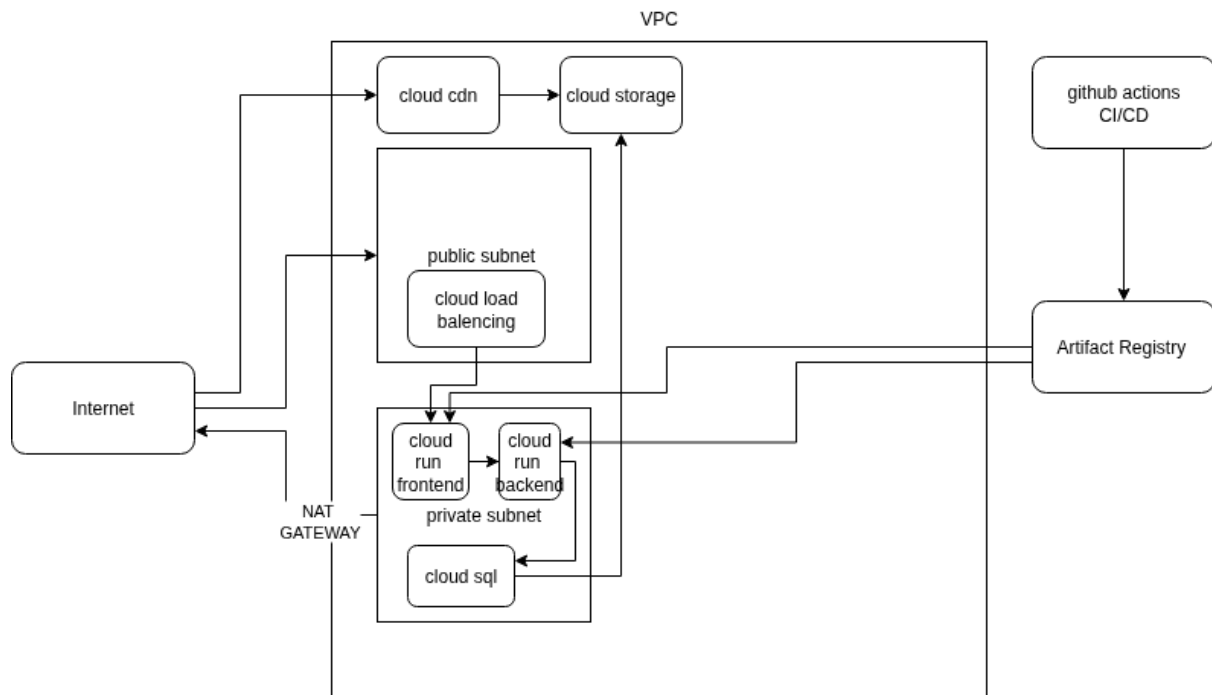
Sistem Informasi Geografis (GIS) adalah sistem komputer yang dirancang untuk menangkap, menyimpan, memanipulasi, menganalisis, mengelola, dan menyajikan semua jenis data geografis. Pengelolaan data spasial pada arsitektur pendukung aplikasi ini dilakukan menggunakan PostGIS, sebuah ekstensi pada PostgreSQL. Ekstensi ini memungkinkan penyimpanan tipe data geometri (*Latitude/Longitude*) secara efisien, serta memungkinkan kueri spasial langsung di tingkat basis data.

BAB III

IMPLEMENTASI

3.1 Perancangan Arsitektur Sistem

Berikut adalah rancangan arsitektur sistem yang akan dilakukan pada layanan cloud



Seluruh infrastruktur dibungkus dalam sebuah *Virtual Private Cloud* (VPC). Jaringan ini memisahkan layanan menjadi dua zona: *Public Subnet* untuk komponen yang berhadapan langsung dengan internet, dan *Private Subnet* yang sepenuhnya terisolasi dari IP Publik. Sebuah NAT Gateway diimplementasikan untuk memberikan akses internet searah bagi layanan di *Private Subnet* (misalnya untuk mengunduh pembaruan *package*) tanpa membuka celah akses masuk dari pihak luar.

Akses masuk dari pengguna (Internet) tidak langsung menyentuh *server* aplikasi, melainkan diterima oleh Cloud Load Balancing di *Public Subnet*. Load Balancer ini mendistribusikan trafik masuk menuju layanan Cloud Run Frontend dan Cloud Run Backend yang berjalan di dalam *Private Subnet*. Penggunaan Cloud Run memastikan aplikasi berjalan di atas layanan *container* yang terkelola (*managed container service*)

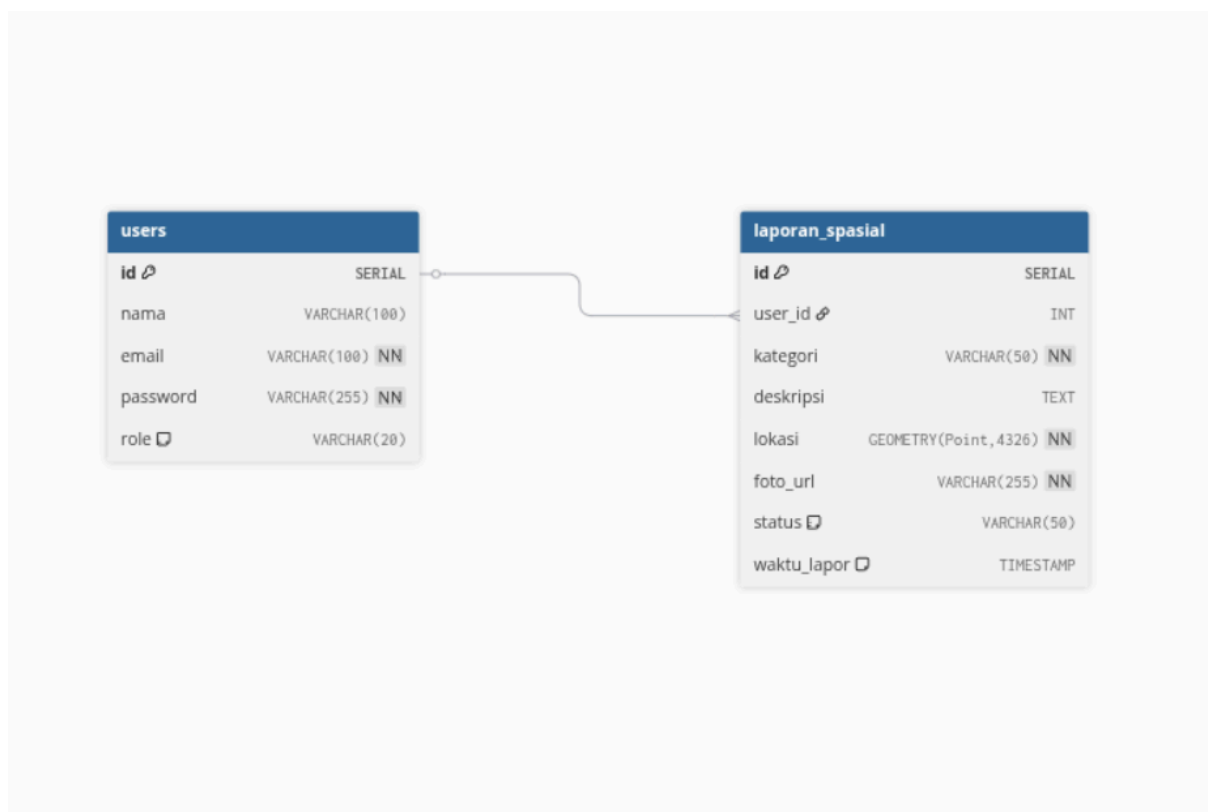
Seluruh logika bisnis dan pemrosesan data dilakukan oleh Cloud Run Backend. Backend terhubung secara aman ke basis data relasional Cloud SQL yang terletak di zona *Private Subnet*. Ketika pengguna mengunggah foto bukti laporan melalui sistem, Cloud Run Backend akan meneruskan dan menyimpan berkas fisik tersebut ke dalam layanan *object storage* Cloud Storage.

Permintaan terhadap foto bukti pelaporan atau aset statis lainnya dari internet akan dialihkan melalui Cloud CDN (*Content Delivery Network*). Cloud CDN bertugas menyajikan *cache* konten ke pengguna dengan latensi rendah.

Ketika terjadi perubahan kode sumber, GitHub Actions (CI/CD) akan menjalankan proses *build* untuk membuat *image* Docker baru. *Image* tersebut kemudian diunggah (*push*) ke layanan repositori Artifact Registry. Setelah tersimpan, *pipeline* akan otomatis memicu proses *deployment* ke layanan Cloud Run agar sistem selalu menggunakan versi aplikasi terbaru.

3.2 Perancangan Database

Berikut adalah perancangan relasi basis data sistem



Berdasarkan gambar ERD di atas, basis data sistem terdiri dari dua tabel utama yang saling berelasi, yaitu tabel *users* dan tabel *laporan_spasial*.

1. Tabel *users* (Entitas Pengguna): Tabel ini berfungsi untuk menyimpan data profil dan kredensial autentikasi pengguna sistem. Tabel ini terdiri dari:
 - *id*: Bertindak sebagai *Primary Key* dengan tipe data SERIAL (auto-increment) untuk memberikan identifikasi unik pada setiap pengguna.
 - *nama*: Menyimpan nama lengkap pengguna dengan batas 100 karakter (VARCHAR(100)).
 - *email*: Menyimpan alamat surel pengguna yang wajib diisi (NN/NOT NULL) dan dibatasi 100 karakter (VARCHAR(100)). Email ini digunakan sebagai *username* saat proses *login*.
 - *password*: Menyimpan kata sandi pengguna yang telah dienkripsi (di-*hash*). Wajib diisi (NN) dan dialokasikan hingga 255 karakter (VARCHAR(255)) untuk mengakomodasi panjang *hash* bcrypt.
 - *role*: Menyimpan peran pengguna dalam sistem (misalnya: 'masyarakat' atau 'admin') dengan batasan 20 karakter (VARCHAR(20)).
2. Tabel *laporan_spasial* (Entitas Pelaporan): Tabel ini adalah inti dari fungsionalitas aplikasi, menyimpan detail laporan pengaduan beserta titik koordinat geografisnya. Tabel ini terdiri dari:
 - *id*: Bertindak sebagai *Primary Key* dengan tipe SERIAL untuk identifikasi unik setiap laporan.
 - *user_id*: Bertindak sebagai *Foreign Key* dengan tipe INT yang merujuk pada *id* di tabel *users*. Kolom ini menetapkan relasi *One-to-Many*, di mana satu pengguna dapat membuat banyak laporan.
 - *kategori*: Menyimpan jenis pelaporan (misalnya: 'infrastruktur_rusak', 'sampah_liar') yang wajib diisi (NN) dengan panjang maksimal 50 karakter (VARCHAR(50)).
 - *deskripsi*: Menyimpan uraian detail mengenai laporan dengan tipe data TEXT untuk menampung kalimat panjang.
 - *lokasi*: Merupakan kolom krusial yang menyimpan titik geografis laporan. Kolom ini menggunakan tipe data spasial GEOMETRY(Point, 4326) dari PostGIS, yang merepresentasikan koordinat titik (longitude dan latitude)

menggunakan sistem referensi spasial WGS 84 (SRID 4326). Kolom ini wajib diisi (NN).

- foto_url: Menyimpan tautan atau *path* menuju berkas foto bukti pelaporan yang tersimpan di Cloud Storage. Wajib diisi (NN) dengan batas 255 karakter (VARCHAR(255)).
- status: Menyimpan status penanganan laporan saat ini (contoh: 'menunggu', 'diproses', 'selesai') dengan tipe VARCHAR(50).
- waktu_lapor: Mencatat stempel waktu (TIMESTAMP) secara presisi kapan laporan tersebut dibuat oleh pengguna.

BAB IV

IMPLEMENTASI

4.1 Kontenerisasi aplikasi

Melakukan Kontenerisasi pada frontend dan backend, lalu melakukan pengujian lokal.

4.1.1 Kontainerisasi Backend

Membuat dockerfile pada folder backend, dengan membuat file bernama Dockerfile pada folder backend lokal.

```
# 1. Gunakan base image Node.js versi ringan (Alpine) untuk menghemat ukuran file
FROM node:20-alpine

# 2. Set direktori kerja di dalam container
WORKDIR /app

# 3. Salin daftar dependensi terlebih dahulu (package.json dan package-lock.json)
COPY package*.json ./

# 4. Install dependensi (hanya dependensi produksi agar container lebih ringan)
RUN npm install --production

# 5. Salin seluruh source code backend ke dalam direktori kerja container
COPY . .

# 6. Informasikan bahwa container ini akan mendengarkan di port 3000
EXPOSE 3000

# 7. Perintah utama untuk menjalankan aplikasi saat container dihidupkan
CMD ["node", "server.js"]
```

Membuat .Dockerignore pada folder backend lokal

```
node_modules
.env
npm-debug.log
.git
README.md
```

4.1.2 Kontenerisasi Frontend

Membuat nginx.conf pada folder frontend lokal, yang berisikan

```
server {  
    listen 80;  
    location / {  
        root /usr/share/nginx/html;  
        index index.html index.htm;  
  
        try_files $uri $uri/ /index.html;  
    }  
}
```

Membuat dockerfile pada folder frontend, dengan membuat file bernama Dockerfile pada folder frontend lokal.

```
# Tahap 1: Gunakan versi 'slim' (Debian-based) untuk menghindari bentrok esbuild Vite  
FROM node:20-slim AS builder  
WORKDIR /app  
COPY package*.json ./  
RUN npm install  
COPY . .  
RUN npm run build  
  
# Tahap 2: Tahap Nginx tetap menggunakan Alpine karena sangat aman dan kecil  
FROM nginx:alpine  
RUN rm -rf /usr/share/nginx/html/*  
COPY --from=builder /app/dist /usr/share/nginx/html  
  
# --- INTEGRASI KONFIGURASI NGINX ---  
# Salin file nginx.conf dari lokal ke dalam sistem Nginx di container  
COPY nginx.conf /etc/nginx/conf.d/default.conf  
# -----  
  
EXPOSE 80  
CMD ["nginx", "-g", "daemon off;"]
```

Membuat .Dockerignore pada folder frontend lokal

```
node_modules
.git
dist
README.md
```

4.1.3 Uji Docker secara lokal

Pengujian docker secara lokal pada docker backend, dengan cara menuju pada folder utama yang menyimpan folder backend dan frontend.

```
=> => unpacking to docker.io/library/geolapor-frontend:latest
• agiel-fernanda@agiell-fernanda-IdeaPad-3-14ITL6:~/kuliah/cloud/evaluasi2/frontend$ cd ..
• agiel-fernanda@agiell-fernanda-IdeaPad-3-14ITL6:~/kuliah/cloud/evaluasi2$ ls
arsitektur_sistem.drawio      database.sql                  frontend
arsitektur_sistem.drawio.png  docker-compose.yml          geolapor-backend
• agiel-fernanda@agiell-fernanda-IdeaPad-3-14ITL6:~/kuliah/cloud/evaluasi2$
```

Folder menyimpan sub folder frontend dan geolapor-backend, dan membuat docker-compose.yml berisikan

```
version: '3.8'
services:
  # 1. Container Database Spasial
  db:
    image: postgis/postgis:15-3.3-alpine # Wajib PostGIS untuk kolom GEOMETRY
    container_name: geolapor-db-local
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: password123
      POSTGRES_DB: geolapor
    ports:
      - "5432:5432"
    volumes: # <--- TAMBAHKAN BAGIAN INI
      - ./database.sql:/docker-entrypoint-initdb.d/init.sql

  # 2. Container Backend
  backend:
    image: geolapor-backend
    container_name: backend-test
    ports:
      - "3000:3000"
    environment:
      - DB_HOST=db # Menunjuk ke nama service 'db' di atas
      - DB_PORT=5432
```

```
- DB_USER=admin
- DB_PASSWORD=password123
- DB_NAME=geolapor
- JWT_SECRET=rahasia_anda
depends_on:
  - db
```

3. Container Frontend

frontend:

```
image: geolapor-frontend
container_name: frontend-test
ports:
  - "8080:80"
```

Pengujian docker secara lokal dilakuaka pada folder yang menyimpan
docker-compose.yml

```
docker compose up -d
```

Melakuakn pengujian pada port frontend docker lokal

```
# 3. Container Frontend
▷Run Service
frontend:
  image: geolapor-frontend
  container_name: frontend-test
  ports:
    - "8080:80"
```

dengan meunuju <http://localhost:8080>, dan melakukan semua fitur.

4.2 Setup Arsitektur System GCP

Membuat environment untuk inialisais lingkungan gcp

```
export PROJECT_ID="vocal-pad-493809-a2"
export REGION="asia-southeast2"
export VPC_NAME="geolapor-vpc"
export PUBLIC_SUBNET="geolapor-public-subnet"
export PRIVATE_SUBNET="geolapor-private-subnet"
export SQL_INSTANCE="geolapor-db-instance"
export REPO_NAME="geolapor-repo"
export BUCKET_NAME="geolapor-storage-bucket"
```

Menyalakan google API gooogle cloud pada komponen arsitektur

```
gcloud services enable \
  compute.googleapis.com \
  sqladmin.googleapis.com \
  run.googleapis.com \
  artifactregistry.googleapis.com \
  storage.googleapis.com \
  vpcaccess.googleapis.com
```

Membuat jaringan (VPC) public subnet dan private subnet

```
# 1. Membuat VPC
gcloud compute networks create $VPC_NAME --subnet-mode=custom

# 2. Membuat Public Subnet
gcloud compute networks subnets create $PUBLIC_SUBNET \
  --network=$VPC_NAME \
  --region=$REGION \
  --range=10.0.1.0/24

# 3. Membuat Private Subnet
gcloud compute networks subnets create $PRIVATE_SUBNET \
  --network=$VPC_NAME \
  --region=$REGION \
  --range=10.0.2.0/24 \
  --enable-private-ip-google-access
```

Membuat cloud router & NAT gateway

```
# 1. Membuat Cloud Router
gcloud compute routers create geolapor-router \
  --network=$VPC_NAME \
  --region=$REGION

#2. Membuat NAT Gateway
gcloud compute routers nats create geolapor-nat \
  --router=geolapor-router \
  --region=$REGION \
  --auto-allocate-nat-external-ips \
  --nat-all-subnet-ip-ranges
```

Persiapan VPC peering database ke jaringan, tanpa ip publik

```
# 1. Alokasikan blok IP khusus untuk layanan Google terkelola
gcloud compute addresses create google-managed-services-$VPC_NAME \
  --global \
  --purpose=VPC_PEERING \
  --prefix-length=16 \
  --description="Peering IP range for Cloud SQL" \
  --network=$VPC_NAME

# 2. Hubungkan VPC Anda dengan jaringan layanan Google
gcloud services vpc-peerings connect \
  --service=servicenetworking.googleapis.com \
  --ranges=google-managed-services-$VPC_NAME \
  --network=$VPC_NAME \
  --project=$PROJECT_ID
```

Membuat database di gcp

```
# 1. Membuat Instans Database PostgreSQL (Hanya dengan Private IP)
gcloud sql instances create $SQL_INSTANCE \
  --database-version=POSTGRES_15 \
  --tier=db-f1-micro \
  --region=$REGION \
  --network=$VPC_NAME \
  --no-assign-ip

# 2. Membuat nama database 'geolapor' di dalam instans tersebut
```

```
gcloud sql databases create geolapor --instance=$SQL_INSTANCE
```

3. Mengatur password untuk user 'postgres' (Bisa Anda ganti password-nya jika mau)

```
gcloud sql users set-password postgres \  
  --instance=$SQL_INSTANCE \  
  --password="password123"
```

Membuat registry di gcp

```
gcloud artifacts repositories create $REPO_NAME \  
  --repository-format=docker \  
  --location=$REGION \  
  --description="Repository Docker untuk Frontend dan Backend GeoLapor"
```

Membuat cloud storage bucket

```
gcloud storage buckets create gs://$BUCKET_NAME \  
  --location=$REGION \  
  --uniform-bucket-level-access
```

Membuat Service Account & Memberikan Peran (Role)

Membuat Service Account

```
gcloud iam service-accounts create github-actions-sa \  
  --description="Service account for GitHub Actions CI/CD" \  
  --display-name="GitHub Actions Deployer"
```

Memberikan akses admin Cloud Run

```
gcloud projects add-iam-policy-binding $PROJECT_ID \  
  --member="serviceAccount:github-actions-sa@$PROJECT_ID.iam.gserviceaccount.com" \  
  --role="roles/run.admin"
```

Memberikan akses push ke Artifact Registry

```
gcloud projects add-iam-policy-binding $PROJECT_ID \  
  --member="serviceAccount:github-actions-sa@$PROJECT_ID.iam.gserviceaccount.com" \  
  --role="roles/artifactregistry.writer"
```

```
# Memberikan izin penggunaan Service Account
gcloud projects add-iam-policy-binding $PROJECT_ID \

--member="serviceAccount:github-actions-sa@$PROJECT_ID.iam.gserviceaccount.com" \
  --role="roles/iam.serviceAccountUser"
```

Konfigurasi Workload Identity Federation

```
# Mengaktifkan API Kredensial IAM
gcloud services enable iamcredentials.googleapis.com

# Membuat Workload Identity Pool
gcloud iam workload-identity-pools create github-pool \
  --location="global" \
  --display-name="GitHub Actions Pool"

# Membuat OIDC Provider untuk GitHub
gcloud iam workload-identity-pools providers create-oidc github-provider \
  --location="global" \
  --workload-identity-pool="github-pool" \
  --display-name="GitHub Actions Provider" \

--attribute-mapping="google.subject=assertion.sub,attribute.actor=assertion.actor,attribute.repository=assertion.repository" \
  --attribute-condition="assertion.repository_owner == 'AgielF'" \
  --issuer-uri="https://token.actions.githubusercontent.com"
```

Menghubungkan Repositori GitHub ke Service Account

```
export PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID
--format="value(projectNumber)")

gcloud iam service-accounts add-iam-policy-binding
github-actions-sa@$PROJECT_ID.iam.gserviceaccount.com \
  --role="roles/iam.workloadIdentityUser" \

--member="principalSet://iam.googleapis.com/projects/$PROJECT_NUMBER/locations/global/workloadIdentityPools/github-pool/attribute.repository/AgielF/environment_report_with_ci-cd"
```

4.3 Membuat github secrest

1. Kredensial Autentikasi (Workload Identity Federation)

Karena kebijakan organisasi Anda melarang kunci JSON, dua rahasia ini adalah penggantinya untuk menghubungkan GitHub dengan GCP secara aman:

- WIF_PROVIDER: Isi dengan *string* lengkap Provider ID yang Anda dapatkan dari terminal (formatnya:
projects/1019871963440/locations/global/workloadIdentityPools/github-pool/providers/github-provider).
- SA_EMAIL: Alamat email *Service Account* yang telah dibuat, yaitu
github-actions-sa@vocal-pad-493809-a2.iam.gserviceaccount.com.

2. Konfigurasi Infrastruktur & Project

Rahasia ini digunakan agar GitHub Actions tahu ke mana harus mengirimkan *image* Docker dan layanan aplikasi Anda:

- PROJECT_ID: ID proyek Google Cloud Anda, yaitu vocal-pad-493809-a2.
- GCS_BUCKET_NAME: Nama *bucket* penyimpanan untuk fitur unggah foto, yaitu
geolapor-storage-bucket.

3. Koneksi Database (Private Subnet)

Sesuai syarat ujian, database harus berada di *private subnet* dan terhubung dengan aman ke aplikasi:

- DB_HOST_PRIVATE: Isi dengan IP Private dari instans Cloud SQL Anda (biasanya berawal dengan 10.x.x.x).
- DB_USER: Nama pengguna database (misalnya: postgres atau admin).
- DB_PASSWORD: Kata sandi database yang Anda atur (misalnya: password123).
- DB_NAME: Nama database yang digunakan, yaitu geolapor.

4. Keamanan Aplikasi (JWT)

- JWT_SECRET: Kunci rahasia untuk proses pembuatan token login (misalnya: `rahasia_super_aman_untuk_token`).

Membuat workflows bergaya monorepo dengan struktur folder

evaluasi2/

├── Platform Lapor online berbasis WEBGIS.pdf

├── arsitektur_sistem.drawio

├── arsitektur_sistem.drawio.png

├── .github/workflows/deploy.yml

├── database.sql

├── geolapor-backend/

| └── (kode backend : src/, config/, dll)

| └── .DockerFile

| └── .Dockerignore

└── frontend/

├── (kode frontend kamu: src/, public/, dll)

└── DockerFile

Isian Deploy.yml

```
name: Deploy GeoLapor to Cloud Run
```

```
on:
```

```
  push:
```

```
    branches: ["main"]
```

```
env:
```

```
  PROJECT_ID: vocal-pad-493809-a2
```

```
  REGION: asia-southeast2
```

```
  GAR_REPO: geolapor-repo
```

```
  BACKEND_IMAGE: geolapor-backend
```

```
  FRONTEND_IMAGE: geolapor-frontend
```

jobs:

deploy:

name: Build and Deploy to GCP

runs-on: ubuntu-latest

permissions:

contents: 'read'

id-token: 'write'

steps:

=====

0. CHECKOUT

=====

- name: Checkout Source Code

uses: actions/checkout@v4

=====

1. AUTHENTICATION (WIF)

=====

- name: Google Auth

id: auth

uses: google-github-actions/auth@v2

with:

workload_identity_provider: \${{ secrets.WIF_PROVIDER }}

service_account: \${{ secrets.SA_EMAIL }}

- name: Set up Cloud SDK

uses: google-github-actions/setup-gcloud@v2

- name: Docker Auth to Artifact Registry

run: gcloud auth configure-docker \${{ env.REGION }}-docker.pkg.dev --quiet

=====

2. BUILD & DEPLOY BACKEND

=====

- name: Build and Push Backend Image

run: |

docker build \

-t \${{ env.REGION }}-docker.pkg.dev/\${{ env.PROJECT_ID }}/\${{ env.GAR_REPO }}/\${{ env.BACKEND_IMAGE }}:\${{ github.sha }} \

./geolapor-backend

docker push \

\${{ env.REGION }}-docker.pkg.dev/\${{ env.PROJECT_ID }}/\${{ env.GAR_REPO }}/\${{ env.BACKEND_IMAGE }}:\${{ github.sha }}

- name: Deploy Backend to Cloud Run

id: deploy-backend

```

uses: google-github-actions/deploy-cloudrun@v2
with:
  service: geolapor-backend
  image: ${{ env.REGION }}-docker.pkg.dev/${{ env.PROJECT_ID }}/${{
env.GAR_REPO }}/${{ env.BACKEND_IMAGE }}:${{ github.sha }}
  region: ${{ env.REGION }}
  env_vars: |
    DB_HOST=${{ secrets.DB_HOST_PRIVATE }}
    DB_PORT=5432
    DB_USER=${{ secrets.DB_USER }}
    DB_PASS=${{ secrets.DB_PASSWORD }}
    DB_NAME=${{ secrets.DB_NAME }}
    JWT_SECRET=${{ secrets.JWT_SECRET }}
    GCS_BUCKET_NAME=${{ secrets.GCS_BUCKET_NAME }}
  flags: |
    --vpc-connector=geolapor-vpc-connector
    --add-cloudsql-instances=${{ env.PROJECT_ID }}:${{ env.REGION
}}:geolapor-db-instance
    --vpc-egress=all-traffic
    --allow-unauthenticated
    --service-account=1019871963440-compute@developer.gserviceaccount.com

# =====
# 3. BUILD & DEPLOY FRONTEND
# =====
- name: Build and Push Frontend Image
  run: |
    docker build \
      --build-arg VITE_API_URL=${{ steps.deploy-backend.outputs.url }}/api \
      --build-arg VITE_CDN_URL=https://storage.googleapis.com/${{
secrets.GCS_BUCKET_NAME }} \
      -t ${{ env.REGION }}-docker.pkg.dev/${{ env.PROJECT_ID }}/${{
env.GAR_REPO }}/${{ env.FRONTEND_IMAGE }}:${{ github.sha }} \
      ./frontend

    docker push \
      ${{ env.REGION }}-docker.pkg.dev/${{ env.PROJECT_ID }}/${{
env.GAR_REPO }}/${{ env.FRONTEND_IMAGE }}:${{ github.sha }}

- name: Deploy Frontend to Cloud Run
  uses: google-github-actions/deploy-cloudrun@v2
  with:
    service: geolapor-frontend
    image: ${{ env.REGION }}-docker.pkg.dev/${{ env.PROJECT_ID }}/${{
env.GAR_REPO }}/${{ env.FRONTEND_IMAGE }}:${{ github.sha }}
    region: ${{ env.REGION }}
    flags: --port=80 --allow-unauthenticated

```

Isian DockerFile backend

```
# 1. Gunakan base image Node.js versi ringan (Alpine) untuk menghemat ukuran file
FROM node:20-alpine

# 2. Set direktori kerja di dalam container
WORKDIR /app

# 3. Salin daftar dependensi terlebih dahulu (package.json dan package-lock.json)
COPY package*.json ./

# 4. Install dependensi (hanya dependensi produksi agar container lebih ringan)
RUN npm install --production

# 5. Salin seluruh source code backend ke dalam direktori kerja container
COPY . .

# 6. Informasikan bahwa container ini akan mendengarkan di port 3000
EXPOSE 3000

# 7. Perintah utama untuk menjalankan aplikasi saat container dihidupkan
CMD ["node", "server.js"]
```

Isian DockerFile frontend

```
# Tahap 1: Gunakan versi 'slim' (Debian-based) untuk menghindari bentrok esbuild Vite
FROM node:20-slim AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .

# --- PERBAIKAN: INTEGRASI VARIABEL BUILD DARI GITHUB ACTIONS ---
# Menangkap argumen yang disuntikkan oleh deploy.yml saat build
ARG VITE_API_URL
ARG VITE_CDN_URL
# Mengubahnya menjadi Environment Variable agar bisa dibaca oleh Vite
ENV VITE_API_URL=$VITE_API_URL
ENV VITE_CDN_URL=$VITE_CDN_URL
# -----

RUN npm run build

# Tahap 2: Tahap Nginx tetap menggunakan Alpine karena sangat aman dan kecil
FROM nginx:alpine
RUN rm -rf /usr/share/nginx/html/*
```

```
COPY --from=builder /app/dist /usr/share/nginx/html

# --- INTEGRASI KONFIGURASI NGINX ---
# Salin file nginx.conf dari lokal ke dalam sistem Nginx di container
COPY nginx.conf /etc/nginx/conf.d/default.conf
# -----

EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Melakukan git push

```
git add .
git commit -m "deploy with ci/cd"
git push origin main
```

Lampiran

Link aplikasi: <https://geolapor-frontend-1019871963440.asia-southeast2.run.app>

Link Source Code: https://github.com/AgielF/envronment_report_with_ci-cd